

UNION-Based SQL Injection — SecureBank Customer Portal

Web Application Penetration Test Report
Author: Pasin Visuttipinate
Conducted in an authorized, isolated lab environment.

1. Executive Summary

A critical SQL injection vulnerability was discovered in the SecureBank Customer Portal. The vulnerability existed in an undocumented internal API endpoint (`/api/v2/customers`) which accepted a numeric `id` parameter. Due to a lack of input sanitization and the use of string concatenation in query construction, an attacker could inject arbitrary SQL through this parameter.

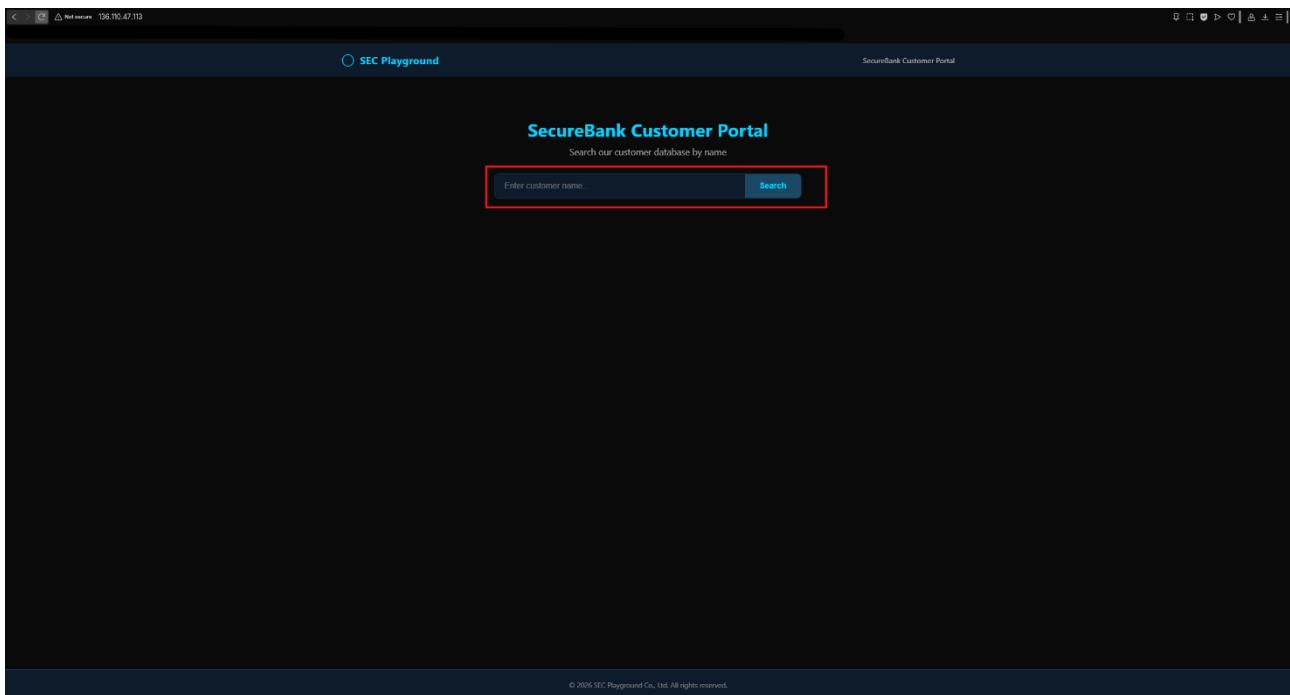
By exploiting this UNION-based SQL injection, complete read access to the underlying SQLite database was achieved. This allowed enumeration of all tables and schemas, and extraction of sensitive data from a `secrets` table (the stored secret value is redacted in this report). The issue was compounded by an information-disclosure flaw: the internal endpoint was referenced in an HTML comment in the public page source.

Field	Detail
Vulnerability	SQL Injection (UNION-based)
Severity	Critical (CVSS 9.8)
Affected endpoint	<code>/api/v2/customers?id=</code>
Database engine	SQLite
Impact	Full database read access; extraction of all data including secrets
Status	Exploited — target value extracted

2. Technical Findings

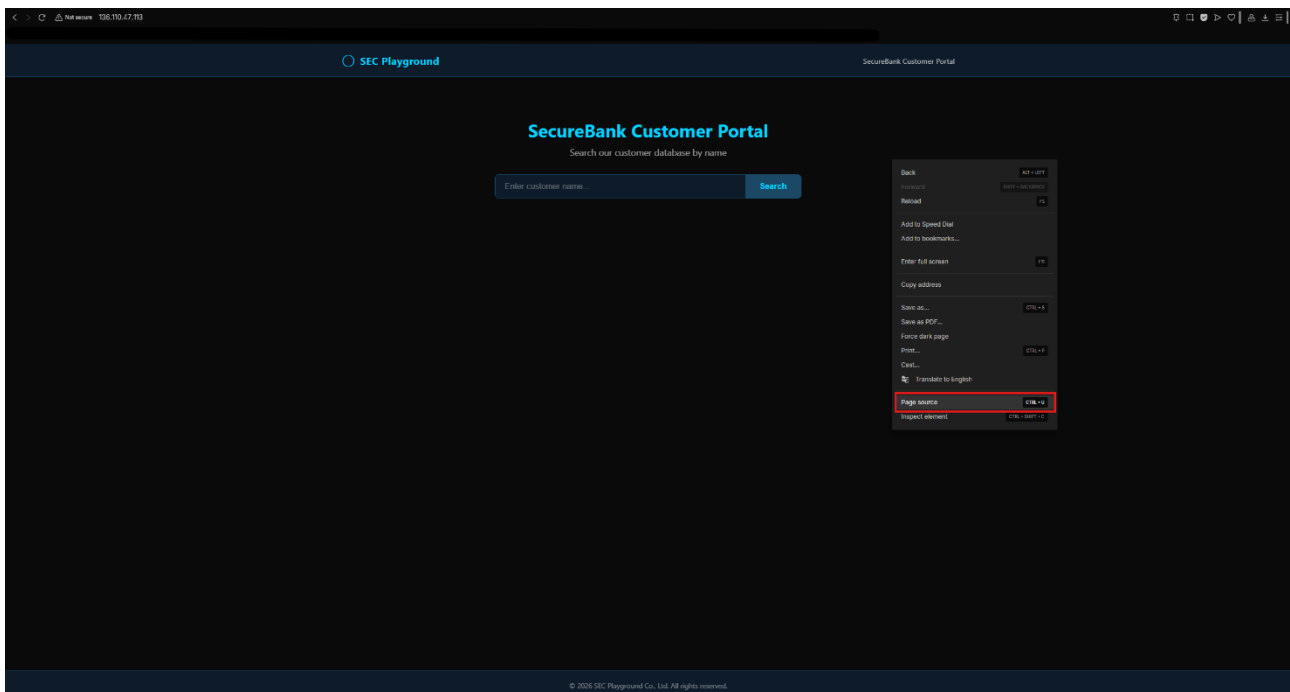
2.1 Reconnaissance — Application Overview

The target is the SecureBank Customer Portal, a web application that searches a customer database by name. The primary search is exposed at `/api/v1/customers?name=`, returning records in JSON with fields `id`, `name`, `email`, and `balance`.



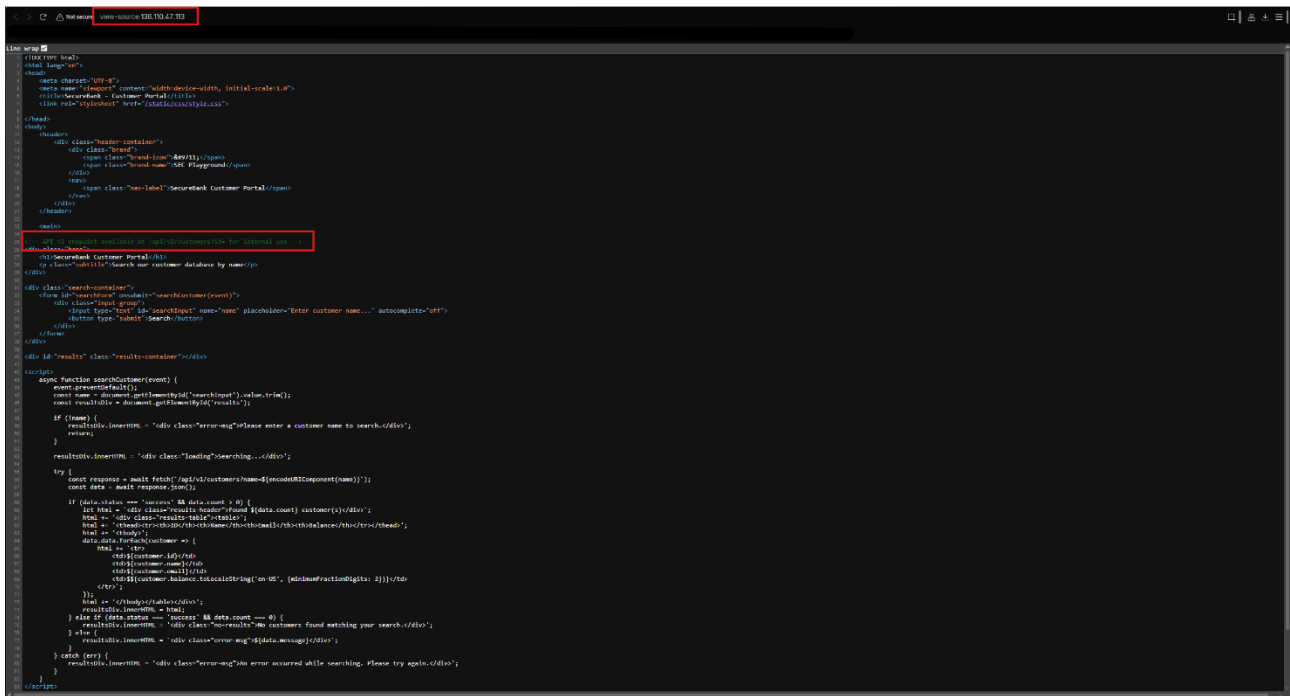
2.2 Information Disclosure — Hidden API Endpoint

Inspecting the HTML page source revealed a comment disclosing an undocumented internal endpoint:



```
<!-- API v2 endpoint available at /api/v2/customers?id= for internal use -->
```

This exposed a secondary API version (`/api/v2/customers`) and that it accepts an `id` parameter. Internal endpoints should never be referenced in client-facing source.

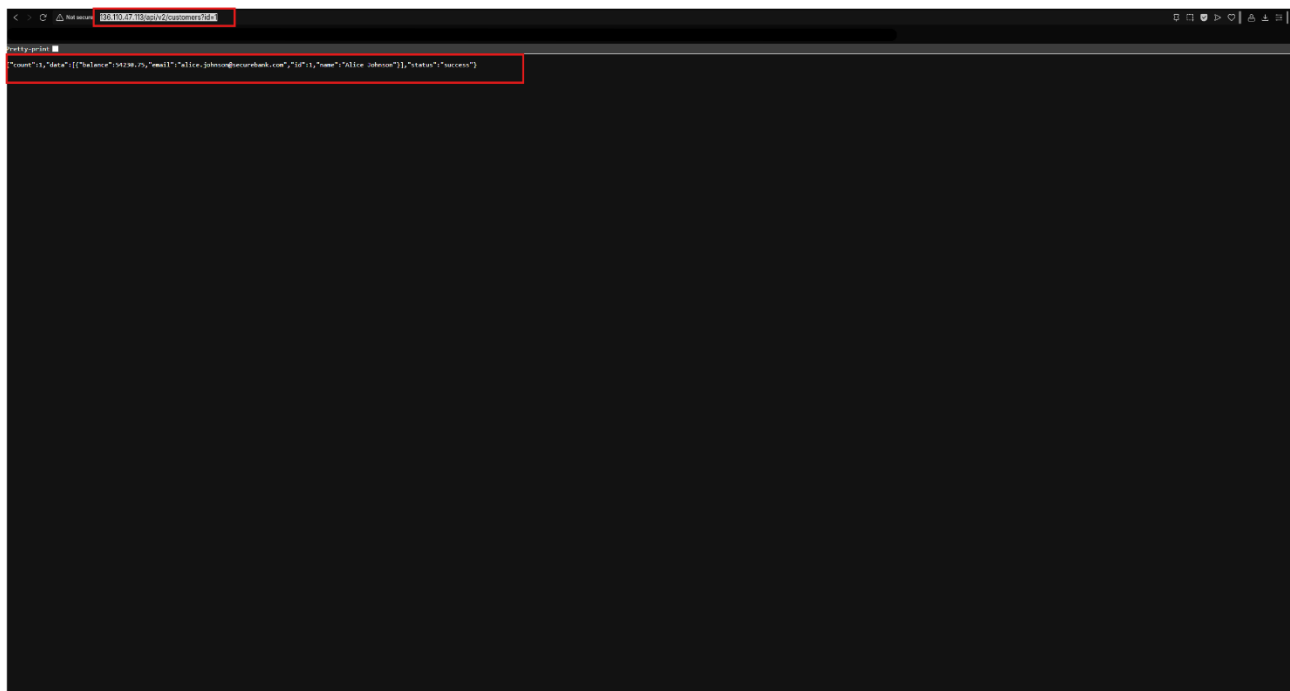


2.3 Confirming the Endpoint

A direct request verified the endpoint was live, publicly accessible, and required no authentication:

```
GET /api/v2/customers?id=1
```

The server returned a valid JSON record.

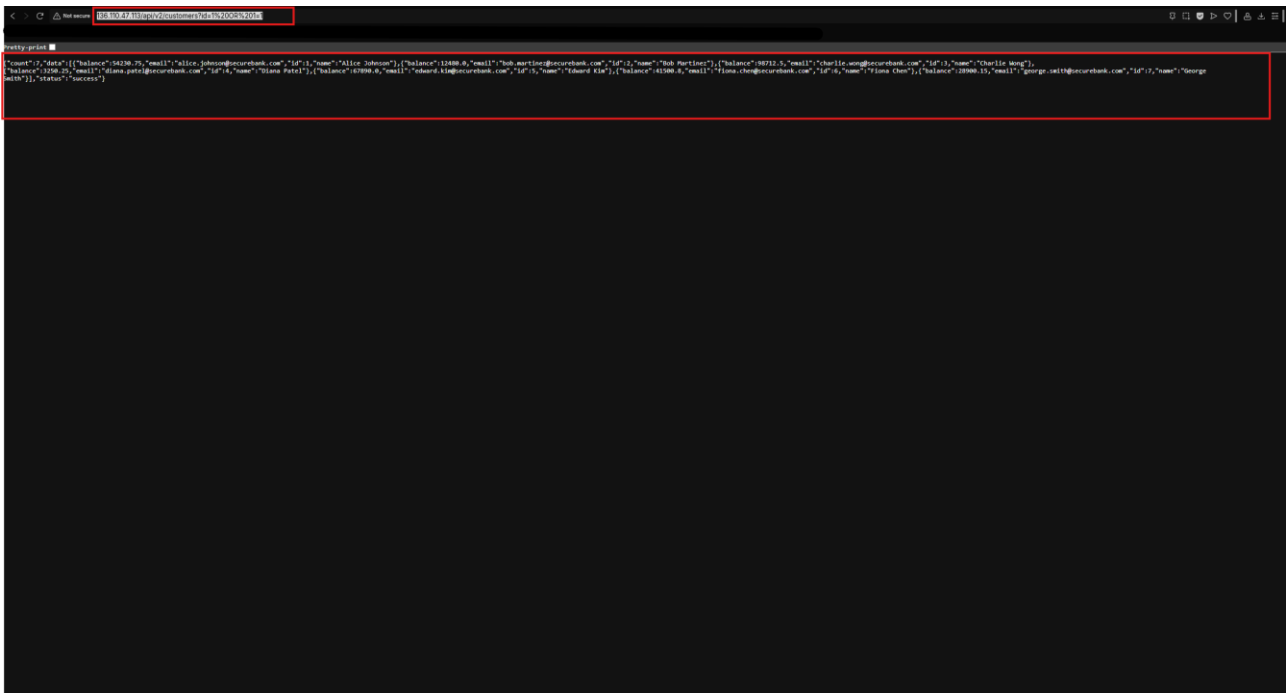


2.4 SQL Injection Confirmation

To test the `id` parameter:

```
/api/v2/customers?id=1 OR 1=1
```

Instead of one record, the server returned all 7 customer records, confirming the `id` value is concatenated directly into the SQL query without parameterization.

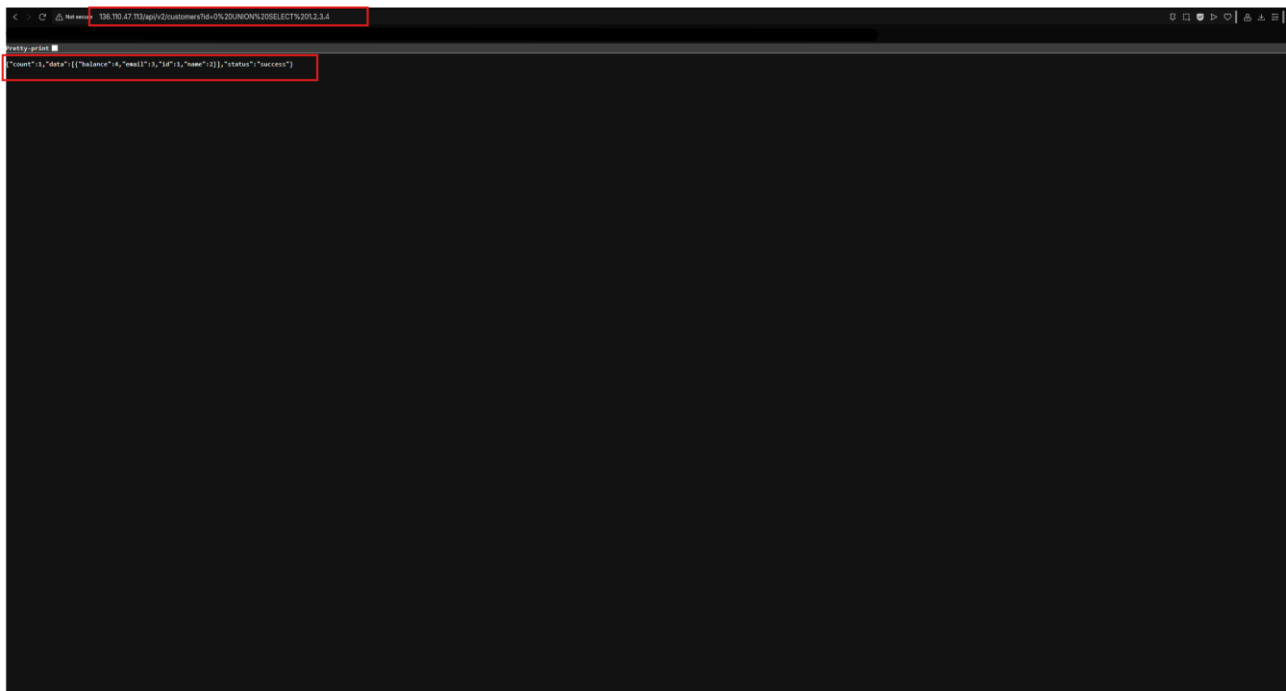


2.5 UNION SELECT — Column Count

A UNION SELECT determined the column count. Using `id=0` ensured the original query returned no rows, so only the injected result displayed:

```
/api/v2/customers?id=0 UNION SELECT 1,2,3,4
```

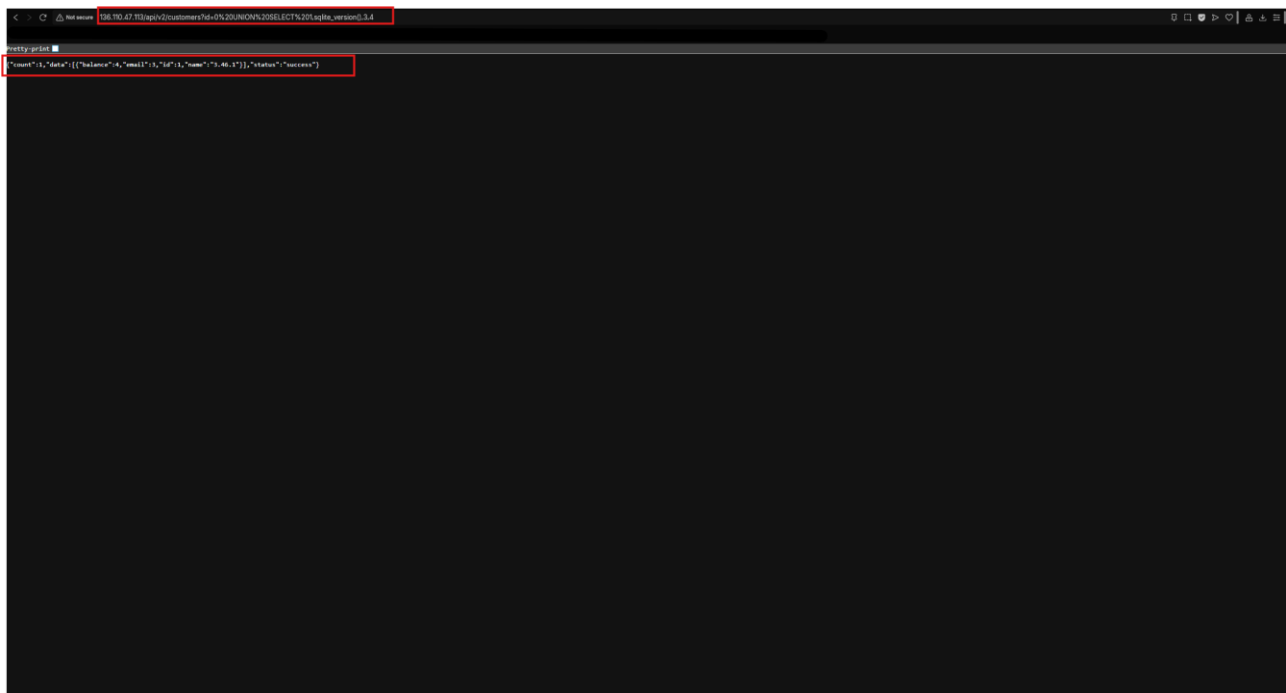
The values 1, 2, 3, 4 mapped to `id`, `name`, `email`, `balance`, confirming 4 columns and the displayed positions.



2.6 Database Engine Identification `version()` and `database()` failed (not MySQL/PostgreSQL). The SQLite-specific function confirmed the engine:

```
/api/v2/customers?id=0 UNION SELECT 1,sqlite_version(),3,4
```

The response returned the SQLite version (3.46.1) in the `name` field.

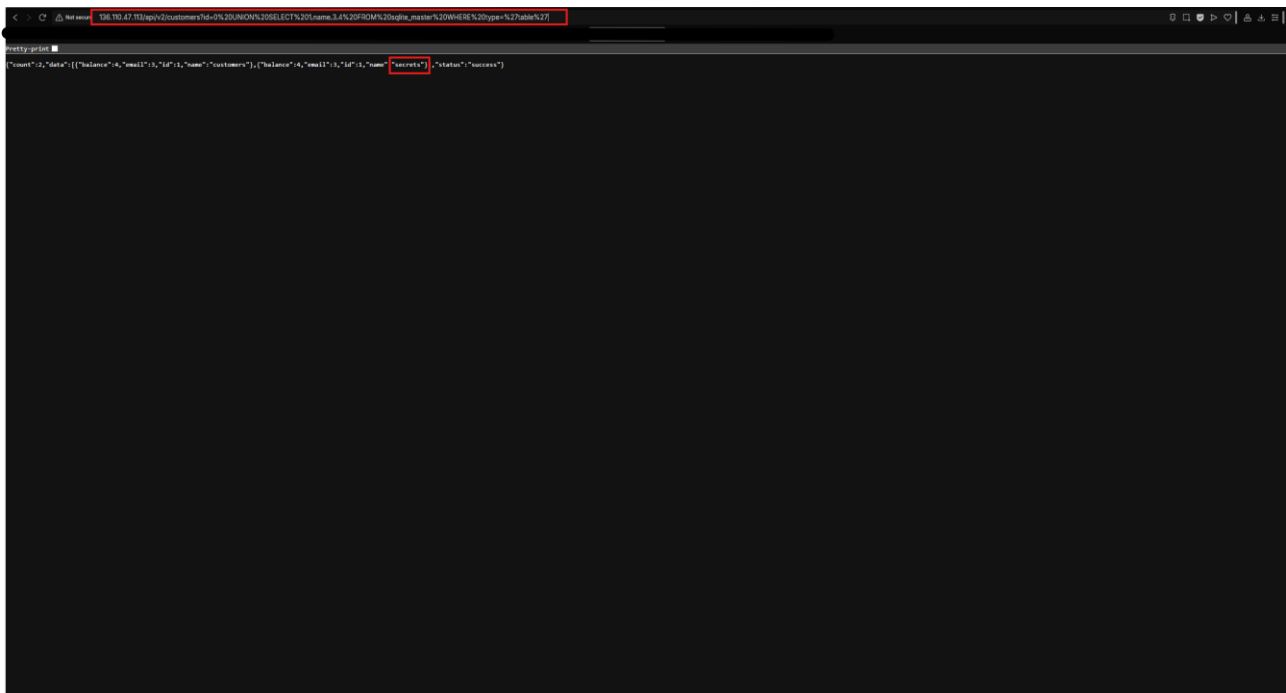


2.7 Table Enumeration

Querying the SQLite system catalogue enumerated all tables:

```
/api/v2/customers?id=0 UNION SELECT 1,name,3,4 FROM sqlite_master WHERE type='table'
```

This revealed two tables: `customers` and a previously unknown `secrets` table.

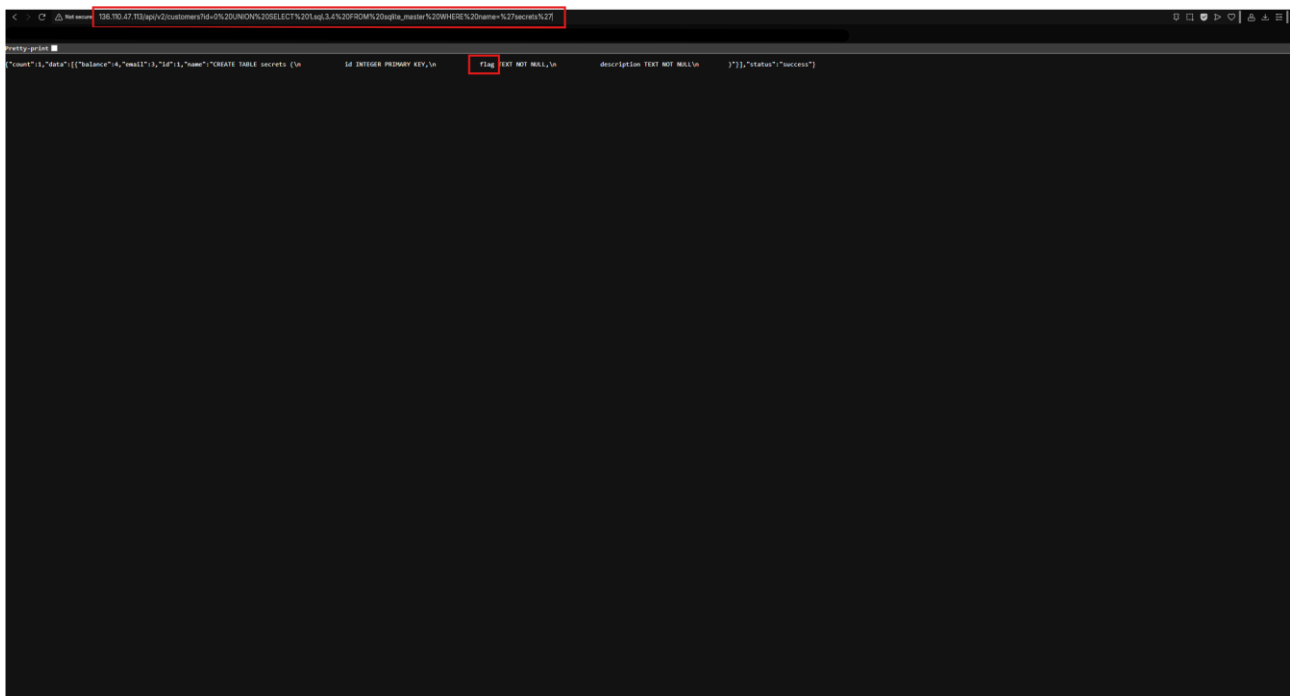


2.8 Schema Extraction

The schema of `secrets` was extracted from the `sql` column of the system catalogue:

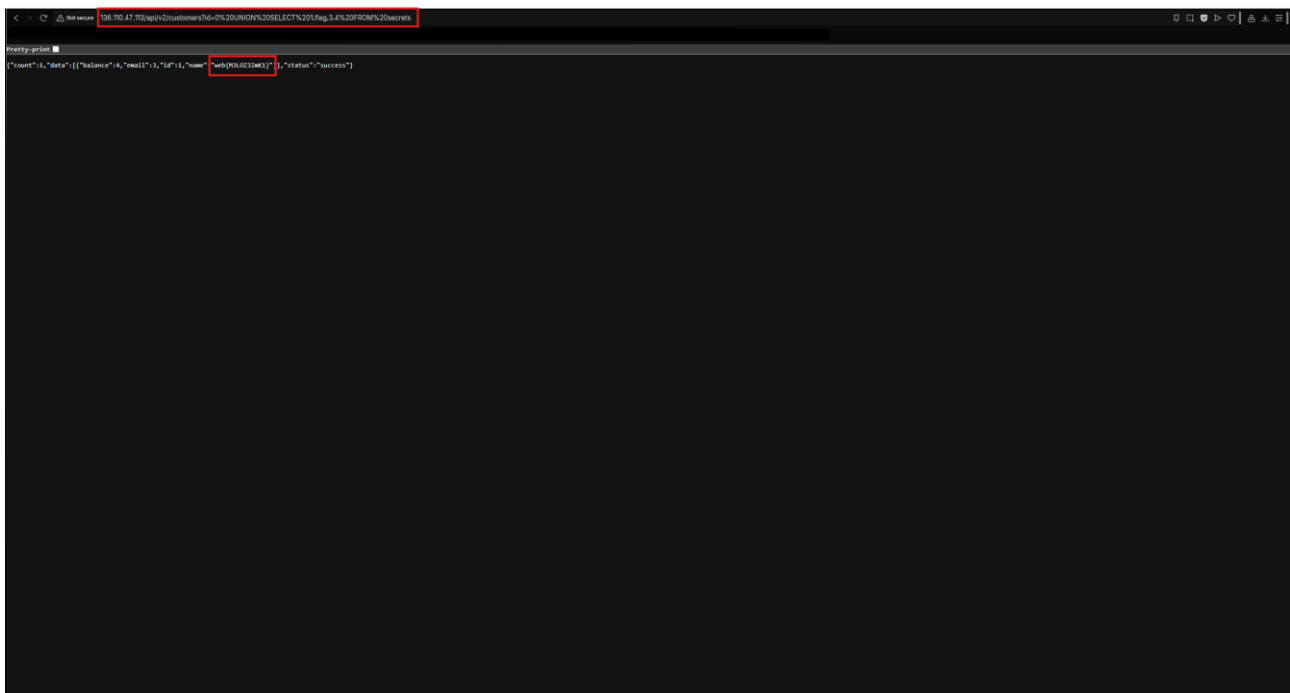
```
/api/v2/customers?id=0 UNION SELECT 1,sql,3,4 FROM sqlite_master WHERE name='secrets'
```

The returned CREATE TABLE statement revealed three columns: `id` (INTEGER PRIMARY KEY), `flag` (TEXT NOT NULL), and `description` (TEXT NOT NULL).



2.9 Outcome

With the table and column known, a final UNION SELECT extracted the target value from the `secrets` table, completing the exploitation and confirming full read access to sensitive data.



3. Severity & Impact Assessment

3.1 Severity Rating

Overall severity: Critical (CVSS 9.8).

Issue	Severity	OWASP category
SQL Injection (UNION-based)	Critical	A03:2021 – Injection
Information disclosure	Medium	A01:2021 – Broken Access Control
Missing authentication on API	Medium	A07:2021 – Authentication Failures

OWASP categories reflect the OWASP Top 10 2021 edition in use at the time of assessment.

3.2 Impact Analysis

- **Complete data breach:** an attacker can read the entire database, including all customer records (names, emails, balances). In a banking application this is catastrophic.
- **Exposure of internal secrets:** the `secrets` table shows that API keys, encryption keys, or configuration secrets stored in the database would be fully exposed.
- **No authentication required:** the endpoint is anonymous, so any internet user could exploit it without credentials.
- **Potential for escalation:** depending on privileges, SQL injection can sometimes reach file-system read or, in other engines, remote code execution; extracted data also enables identity theft and fraud.
- **Regulatory consequences:** a breach of this nature would trigger notification requirements under GDPR, PDPA, and other data-protection laws, with potential fines and reputational damage.

4. Remediation Recommendations

4.1 Immediate Fixes (Critical Priority)

- **Parameterized queries (prepared statements)** — the primary fix. Replace string concatenation:

```
# VULNERABLE: query = "SELECT * FROM customers WHERE id =  
" + user_input # SECURE: query = "SELECT * FROM customers  
WHERE id = ?" cursor.execute(query, (user_input,))
```

- **Input validation:** accept only integer values for `id`; reject non-numeric input before constructing the query.
- **Remove the internal endpoint from public access:** delete it if unused, restrict to internal networks, or protect with authentication and authorization.

4.2 Defense-in-Depth (High Priority)

- **Remove HTML comments containing sensitive information**, with code-review checks before deployment.
- **Implement API authentication and authorization** on all endpoints, especially "internal" ones.
- **Apply least privilege** to the application's database account; the `secrets` table should not be reachable by the web app's DB user.
- **Deploy a WAF** to detect common injection patterns as a secondary (not primary) control.
- **Generic error handling** so database errors do not expose internal detail.
- **Encrypt sensitive data at rest**, so read access alone does not yield usable secrets.